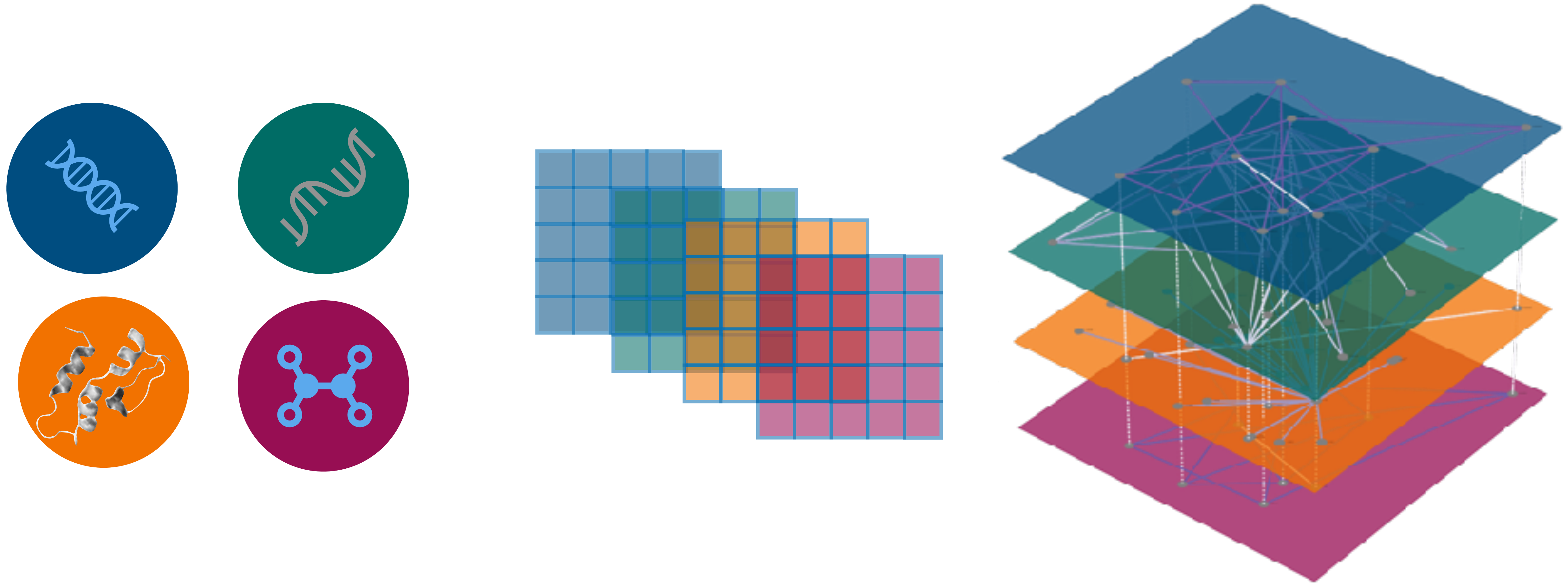


Graph Neural Networks

AI-guided Protein Science

Alberto Santos — Multi-omics Network Analytics (MoNA)

DTU Health Tech
Department of Health Technology

Limitations of Shallow Embeddings

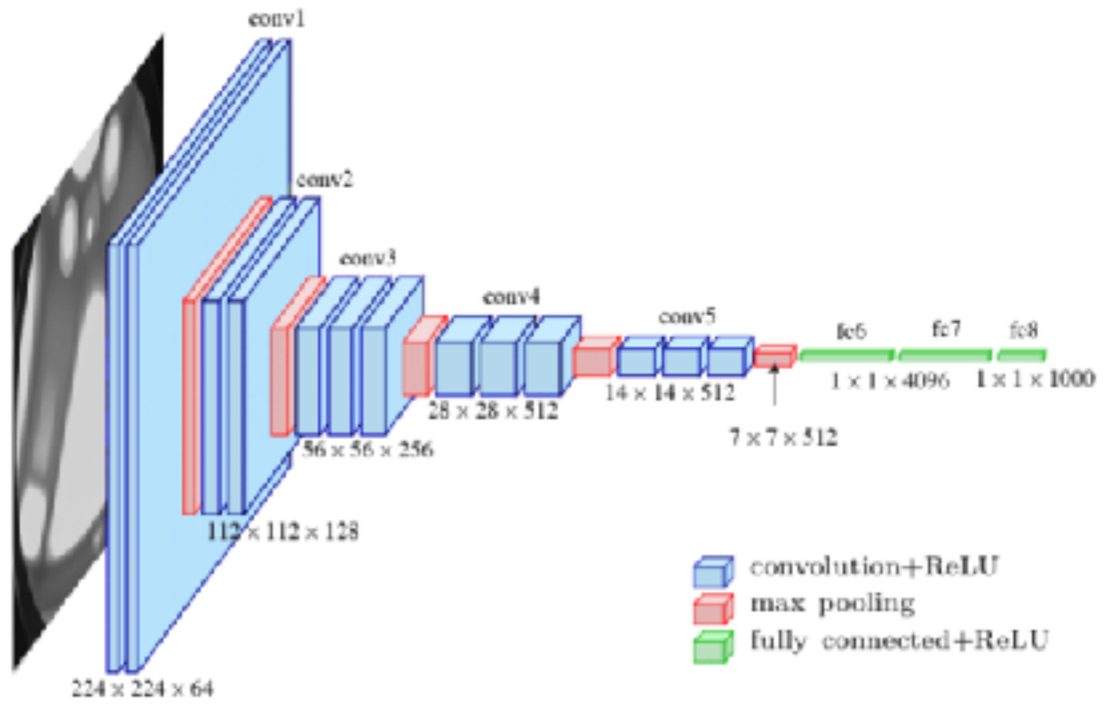
- **Complexity grows** with the number of nodes ($O(|V|)$) — problem with large graphs
- They **do not use node features**
- They are **transductive** — can only generate embeddings for nodes present during the training phase (no generalizable to unseen nodes)

Need More Complex Encoder Models

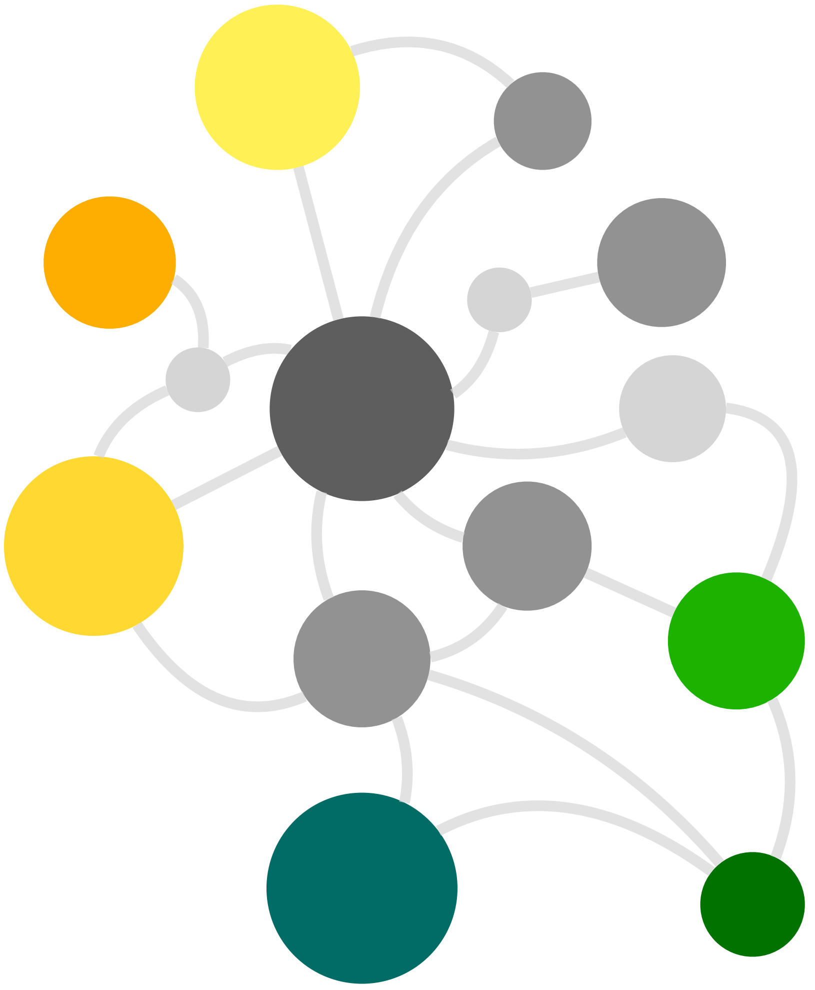
- **Graph Neural Networks** — a deep learning framework for representing graph data
  - Incorporate both local and global context
  - Allows node features (e.g., expression)
  - Inductive

Grid structures

CNNs



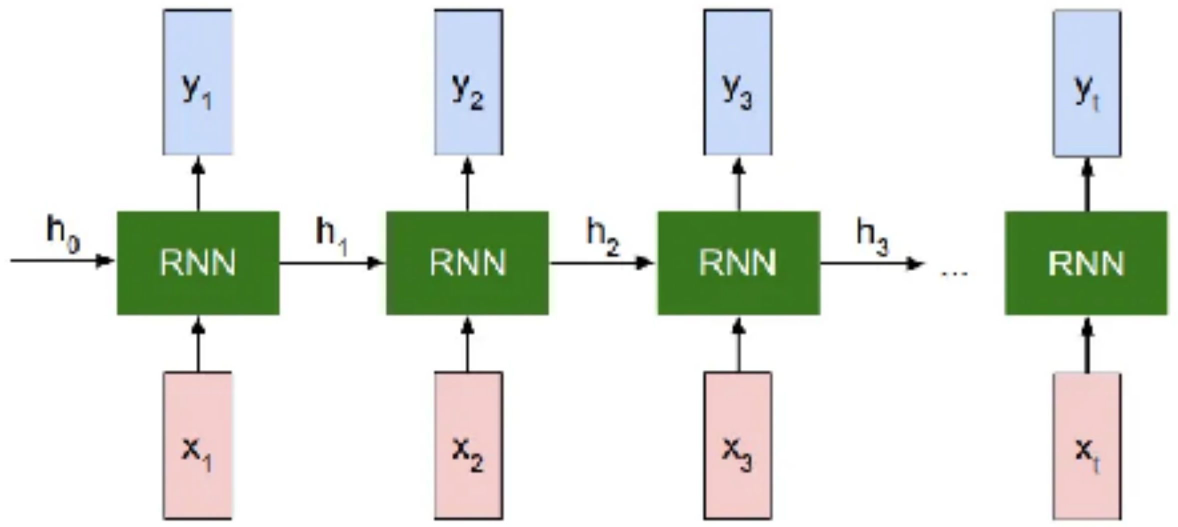
Graph structures



?

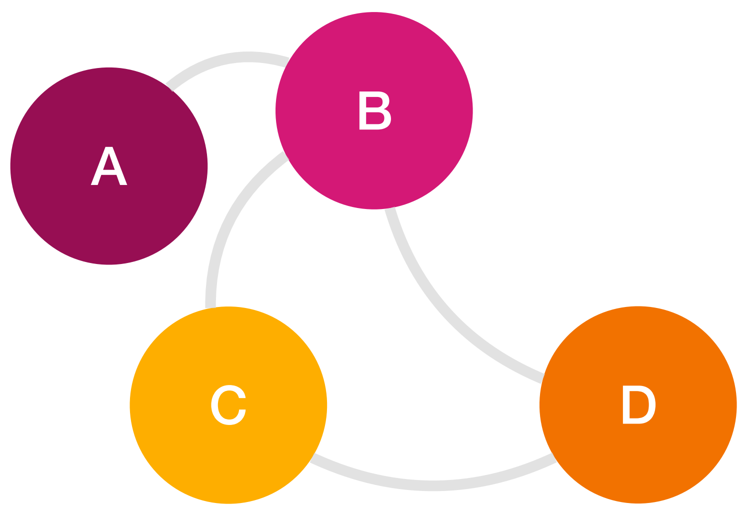
Sequences

RNNs



Using the Adjacency Matrix

- Why not using the Adjacency Matrix ***A***?
  - Assumptions for DL
    - **Permutation invariance**: same output regardless of the order of its input.
    - **Permutation equivariance**: the output reflects the permutation applied to the input.
  - ***A*** has an arbitrary order of nodes



	A	B	C	D
A	0	1	0	0
B	1	0	1	1
C	0	1	0	1
D	0	1	1	0

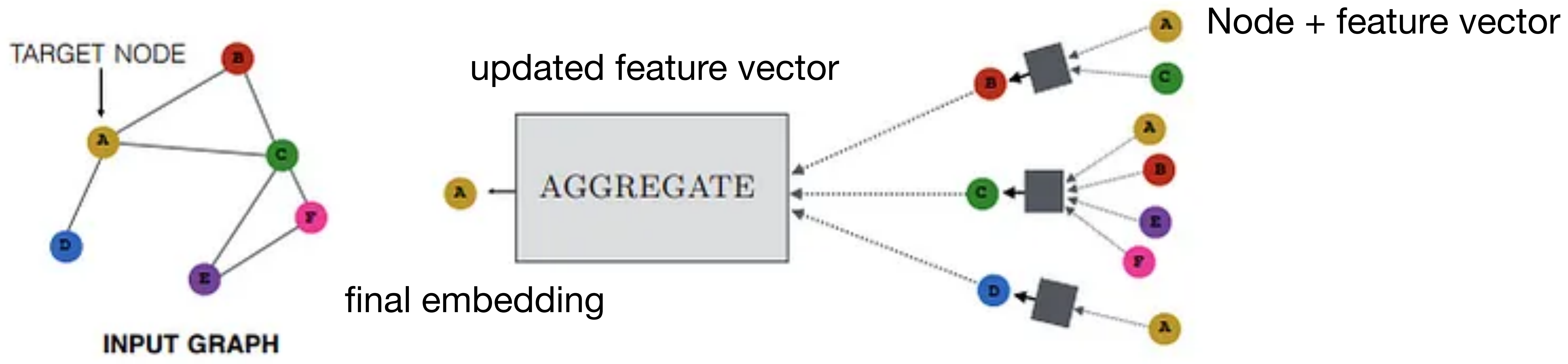
	B	A	D	C
A	1	0	0	0
B	0	1	1	1
C	1	0	1	0
D	1	0	0	1

Neural Message Passing

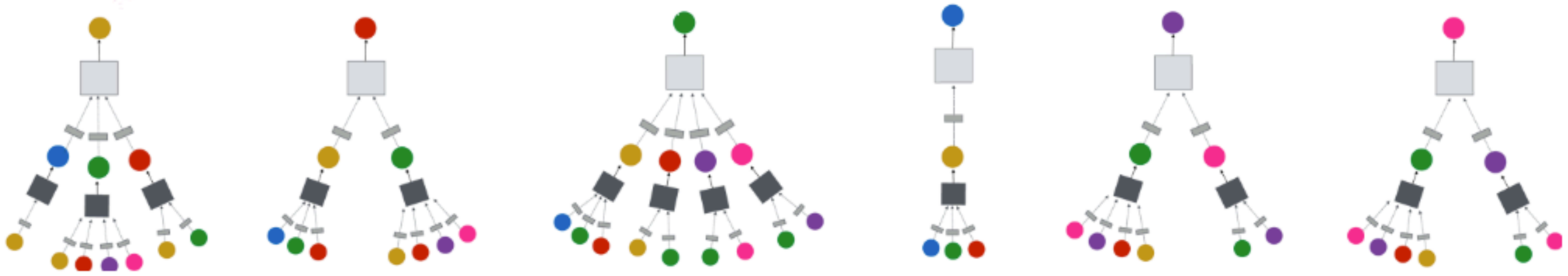
- Enables information exchange and aggregation among nodes capturing the dependencies and interactions among them
- The information collected from neighbours is used to update the representation of the node

At each iteration (layer) of the message passing in a GNN, a hidden embedding $h_u^{(k)}$ is updated for each node *u* based on information gathered from its graph neighbourhood N(u).

As these iterations progress, each node embedding contains more and more global information



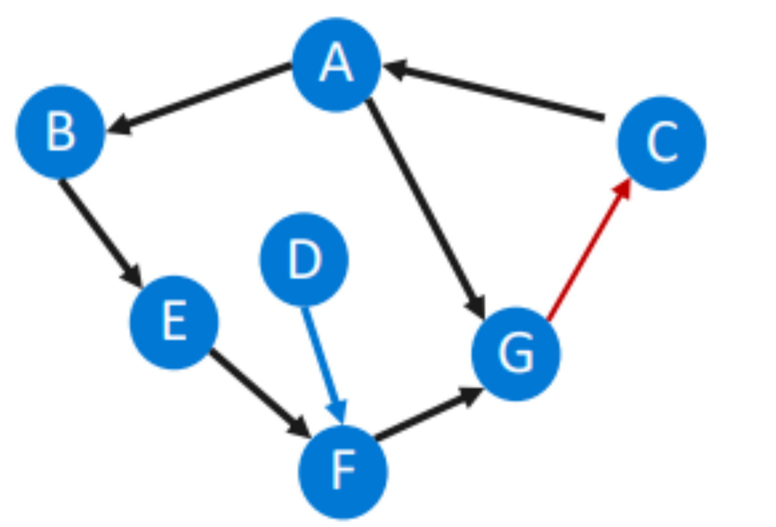
Each node defines a computational graph based on its neighbours



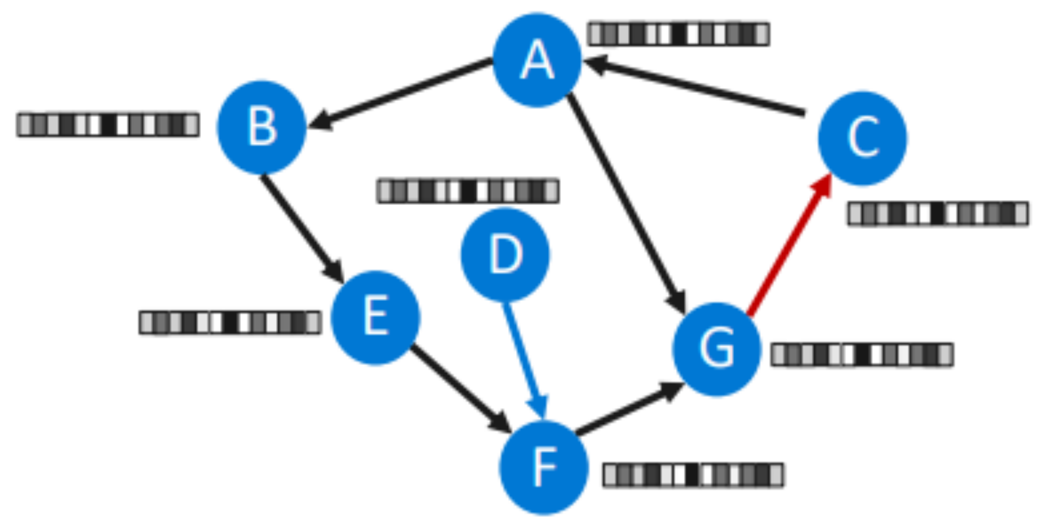
Graph Neural Networks

- A GNN is a neural network that learns relationships on data in a graph. In the image, the problem is represented by an input graph. Each node has an initial vector representation.

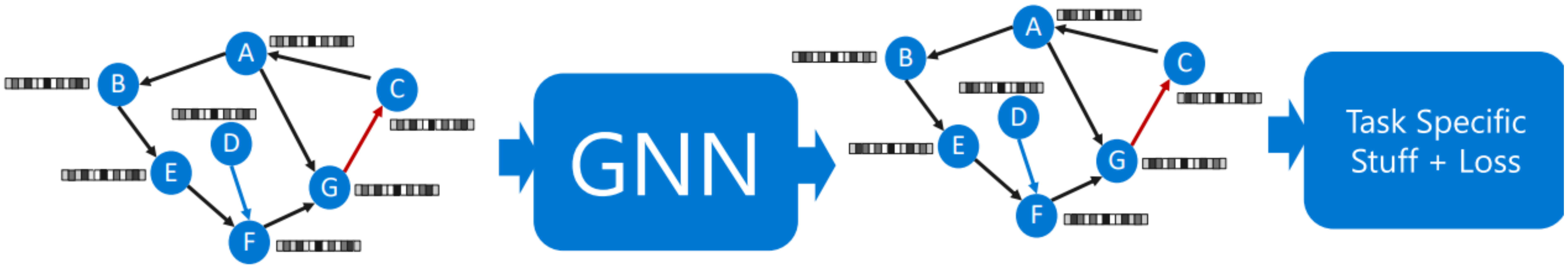
- During training, the GNN updates these node vectors to reflect information that is specific to the task or problem to be solved. It does this through iterative message passing.



Graph Representation of Problem



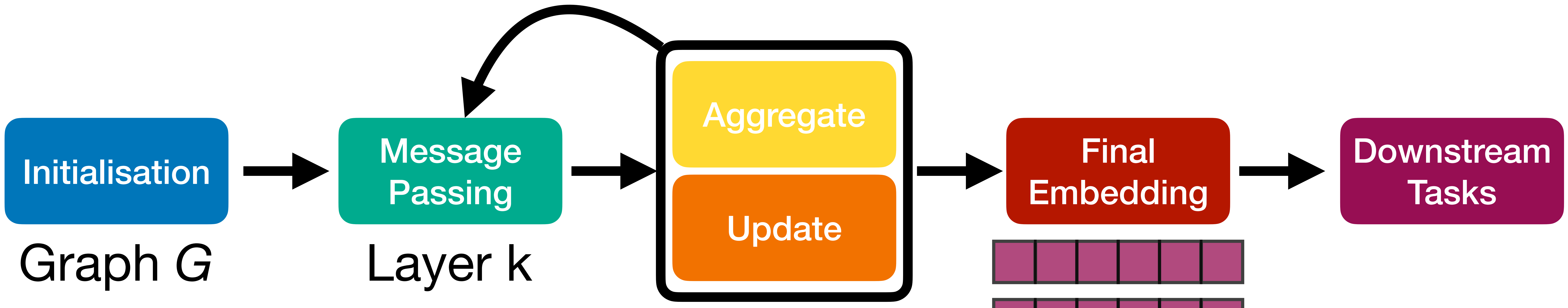
Initial Representation of each node



Initial Representation of each node

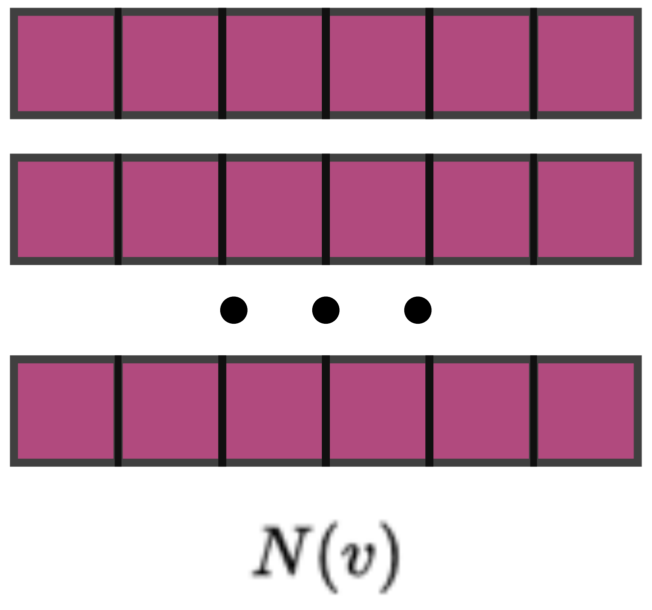
Output Representations of each Node

GNN Workflow



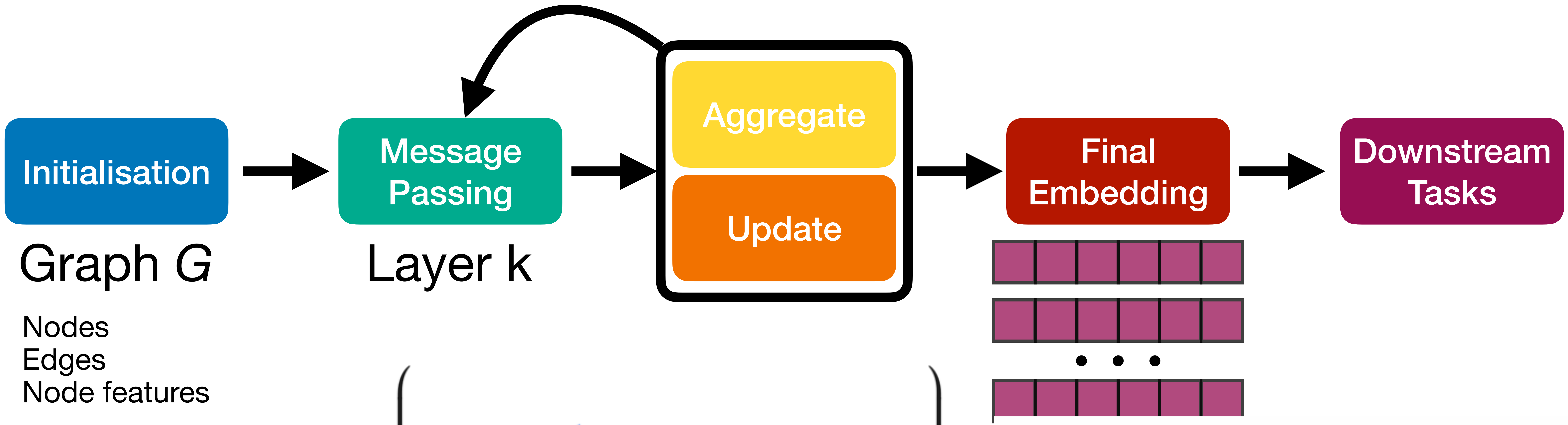
Nodes
Edges
Node features
Edge features

$$\mathbf{h}_i^{(k+1)} = \sigma\left(\sum_{j\in\square(i)} \frac{1}{\sqrt{d_i d_j}} \mathbf{W}^{(k)}\mathbf{h}_j^{(k)}\right)$$

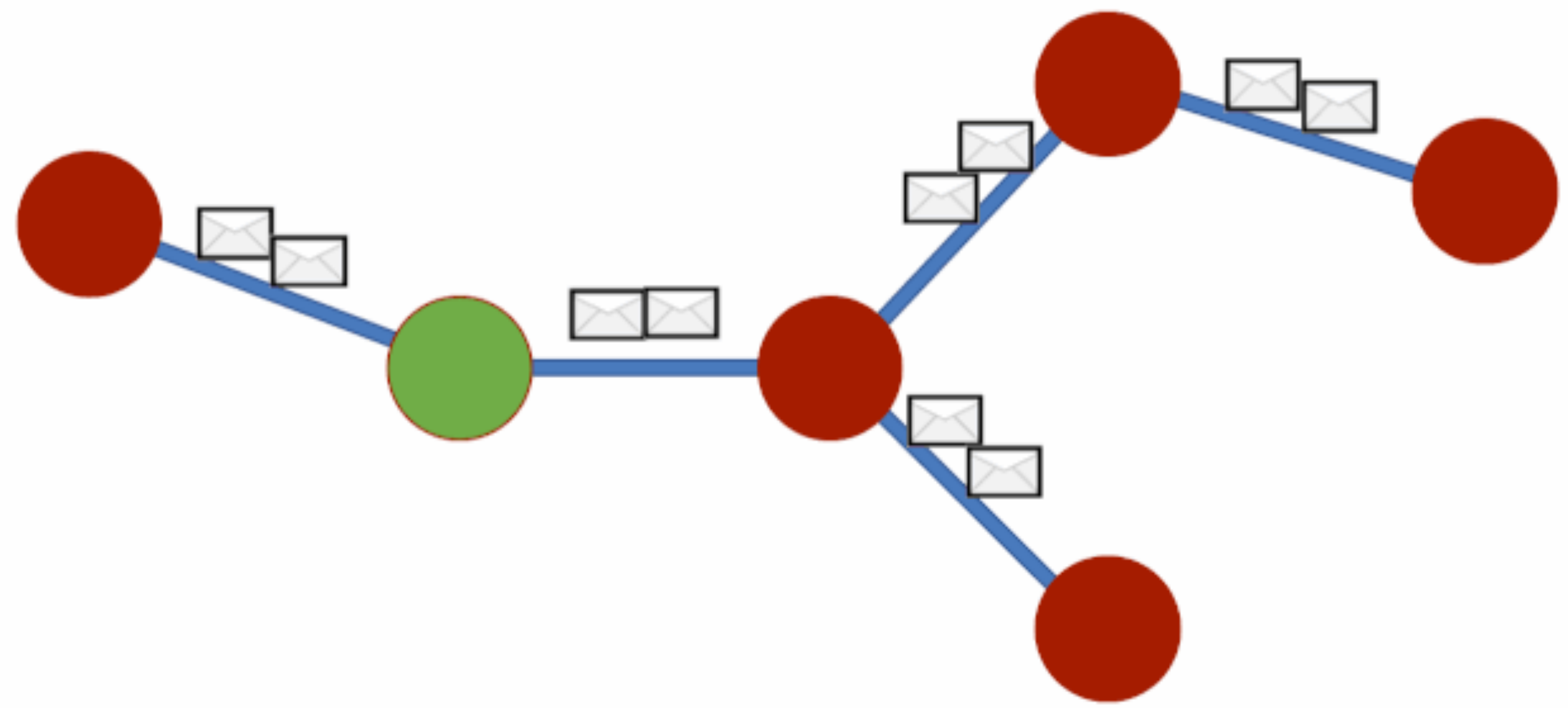


- $\mathbf{h}_i^{(k+1)}$: The updated node embedding for node i at layer $k+1$.
- $\mathbf{h}_j^{(k)}$: The embedding of a neighboring node j at the current layer k.
- $\square(i)$: The set of neighboring nodes of i.
- d_i and d_j: The degrees of nodes i and j, respectively. The term $\frac{1}{\sqrt{d_i d_j}}$ serves as a normalization factor.
- $\mathbf{W}^{(k)}$: The weight matrix for layer k, which is trainable.
- σ: An activation function, such as ReLU, to introduce non-linearity.

GNN Workflow



$$h_A^{(1)} = \sigma\left(\sum_{j\in\square(A)} \underbrace{\frac{1}{\sqrt{d_A d_B}}}_{\text{Normalization}} \underbrace{W^{(0)}}_{\text{Weights}} \underbrace{h_B^{(0)}}_{\text{Neighbor's Features}} \right)$$

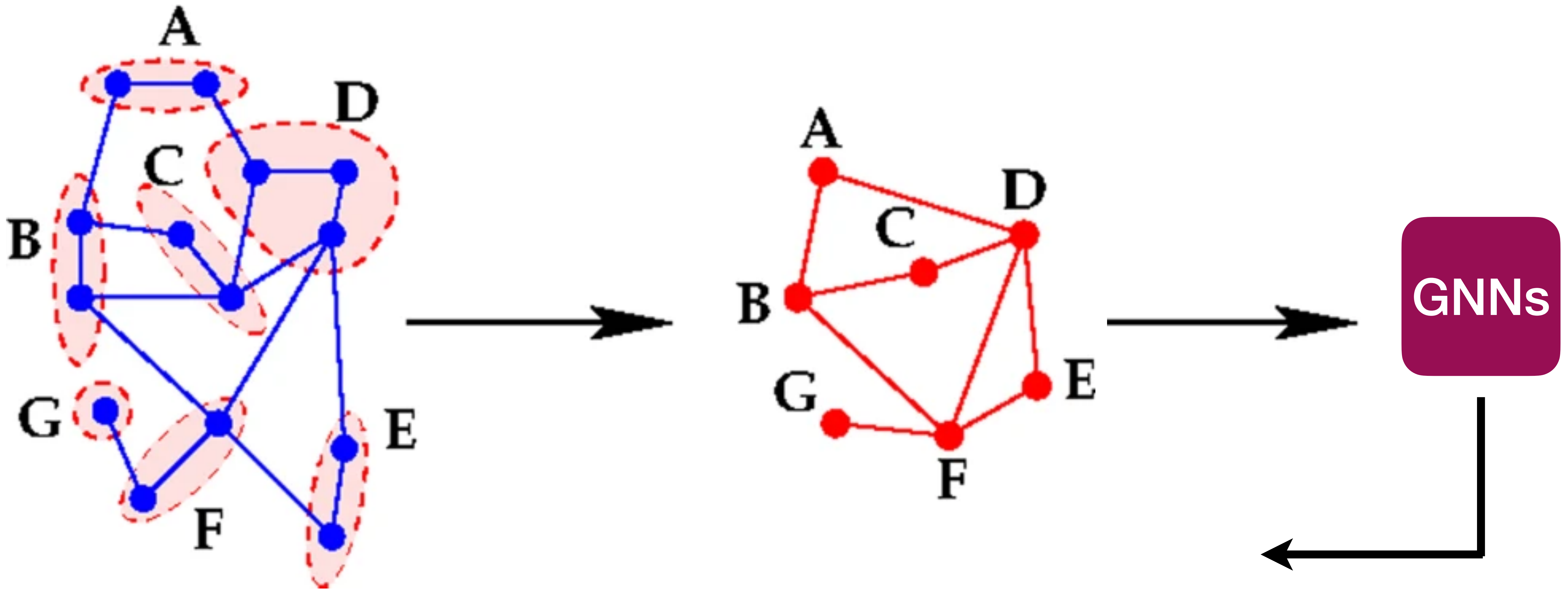


GNN Workflow

Step by step workflow

Graph Pooling

- **Graph-level** tasks — Define a **pooling function** that maps a set of node embeddings to an embedding representing the full graph.
- Strategies:
    - Take the **sum or mean** of the node embeddings
    - Combination of **LSTMs and attention** to pool the node embeddings
    - **Clustering** (e.g., spectral clustering)



Variations of GNNs

- **Graph Convolutional Networks (GCN)**: A popular form of GNN where the aggregation step is similar to a convolution operation on graphs.
- **Graph Attention Networks (GAT)**: Incorporates attention mechanisms, allowing each node to weight the importance of its neighbours differently.
- **GraphSAGE**: A GNN variant that samples a fixed-size neighbourhood for each node to improve scalability for large graphs.

Model	Concept	Best for
GCN	Semi-supervised node classification, graph classification	Simple, dense graphs where all neighbors are generally trustworthy and relevant
GAT	Tasks where **neighbour importance varies** (e.g., node classification with varying relevance of neighbours)	Complex networks where some connections are vital and others may be just noise.
GraphSAGE	**Scalable** learning on **large graphs**, node classification, link prediction	large-scale graphs like knowledge graphs

Generative Models

- **Variational Graph Autoencoders (VGAE)** - an extension of Variational Autoencoders (VAEs) for graphs. It uses a GNN encoder and a decoder that reconstructs the graph structure (adjacency matrix or edge probabilities) using the latent variables.

- **Graph Convolutional GAN (GraphGAN)** - a generative adversarial network (GAN) designed to generate graph-structured data. It uses a generator to create graph structures and a discriminator to differentiate between real and fake graphs.

- **Graph Recurrent Neural Networks (GRNN)** - GRNNs apply RNNs to graphs, generating node sequences or edge sequences iteratively. They can be used for generating dynamic graphs, where nodes and edges evolve over time.

Pay Attention

Over-Smoothing

- If your network has too many layers (usually more than 3 to 5), all node representations become mathematically identical, and your model loses its ability to classify individual nodes.

Over-Squashing

- Crucial information from distant nodes gets entirely squeezed out and lost.

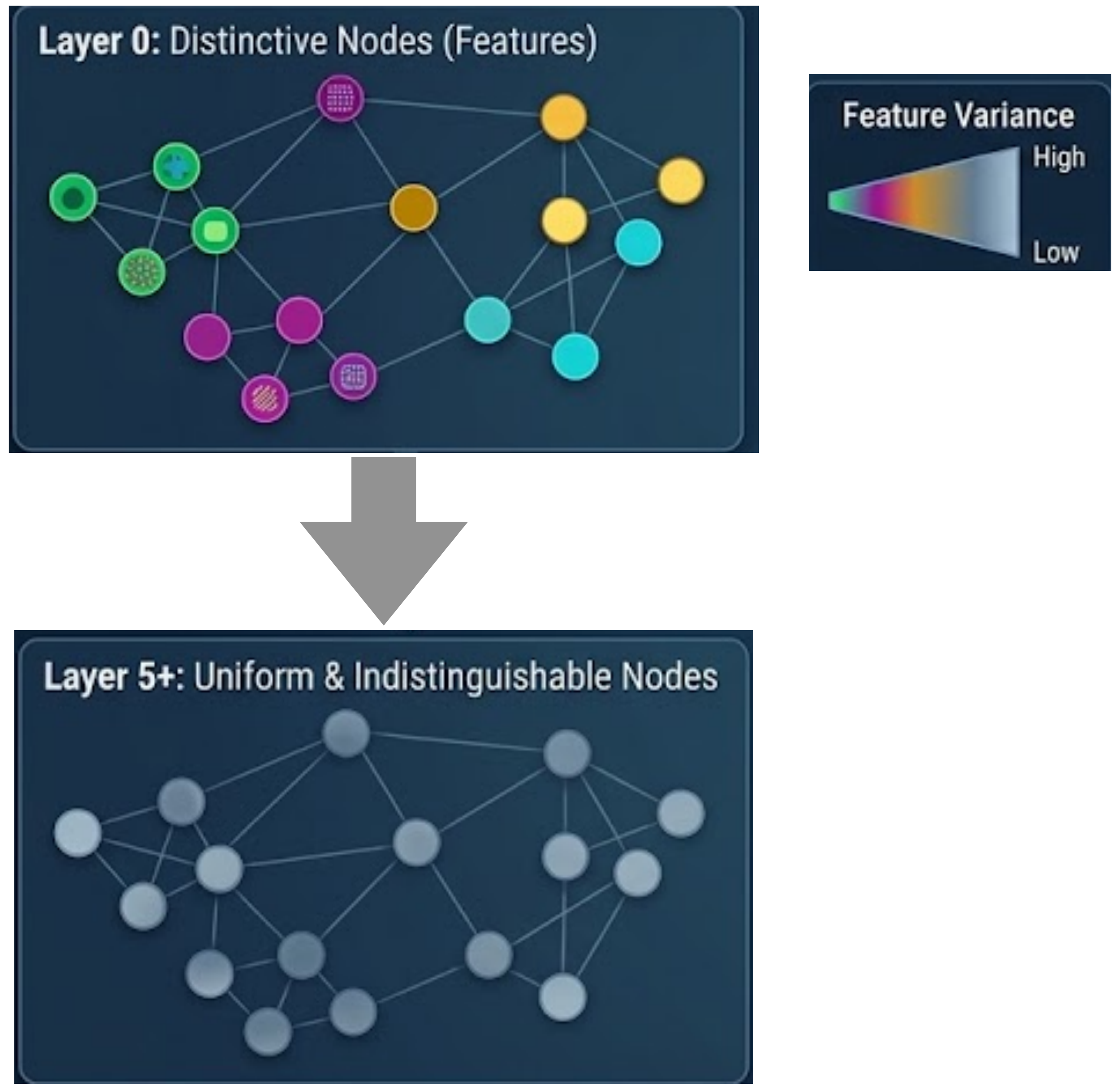
Heterophily

- Most GNNs operate on the assumption of Homophily, however, many real-world graphs exhibit Heterophily.

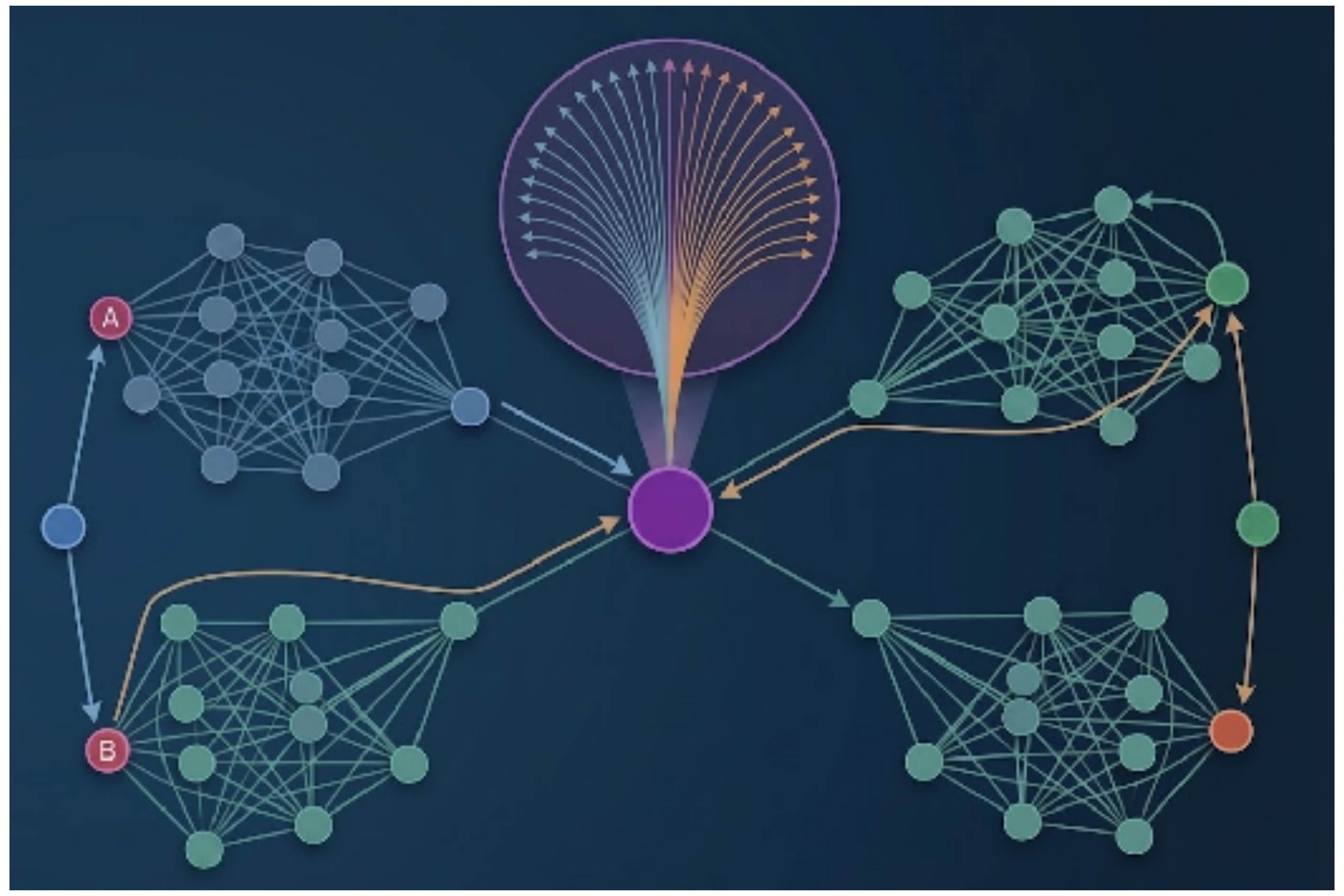
Scalability

- It is difficult to break a graph into"mini-batches". Trying to load a massive graph into GPU memory will cause an Out Of Memory (OOM) error.

Over-Smoothing



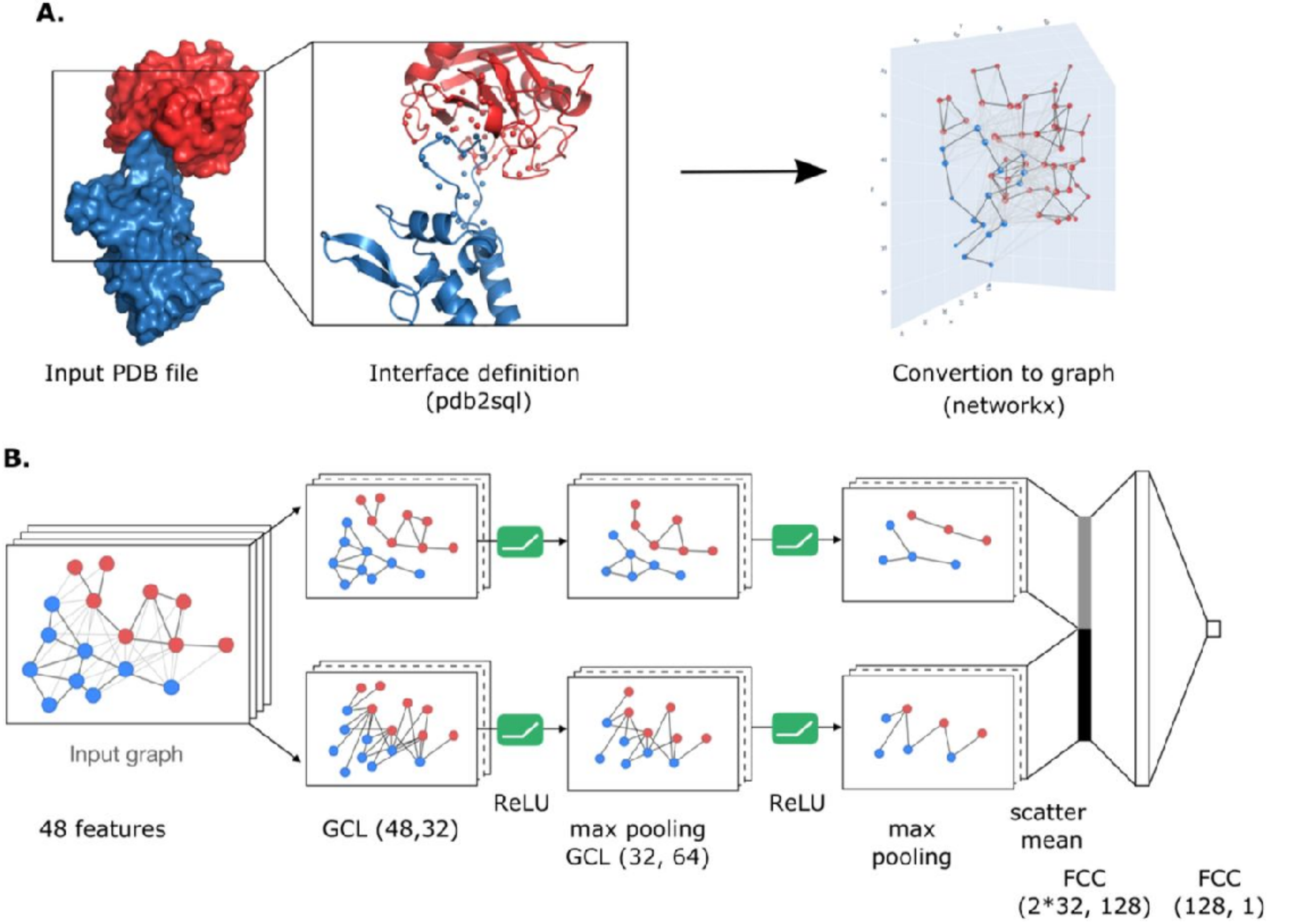
Over-Smoothing



Examples In Protein Design

DeepRank-GNN

- DeepRank-GNN converts protein-protein interfaces into residue-level interaction graphs.
- Contact Maps: Learns the "grammar" of interface residues.
- Scoring: Accurately ranks docked complexes to identify true binders.
- Usage: Critical for assessing candidate designs before experimental testing.



https://github.com/DeepRank/deeprank2 https://doi.org/10.1093/bioinformatics/btac759

GNN Python Libraries

Jraph

https://pytorch-geometric.readthedocs.io/

https://www.dgl.ai/

https://graphneural.network/

https://github.com/google-deepmind/jraph

GNN Python Libraries

https://pytorch-geometric.readthedocs.io/

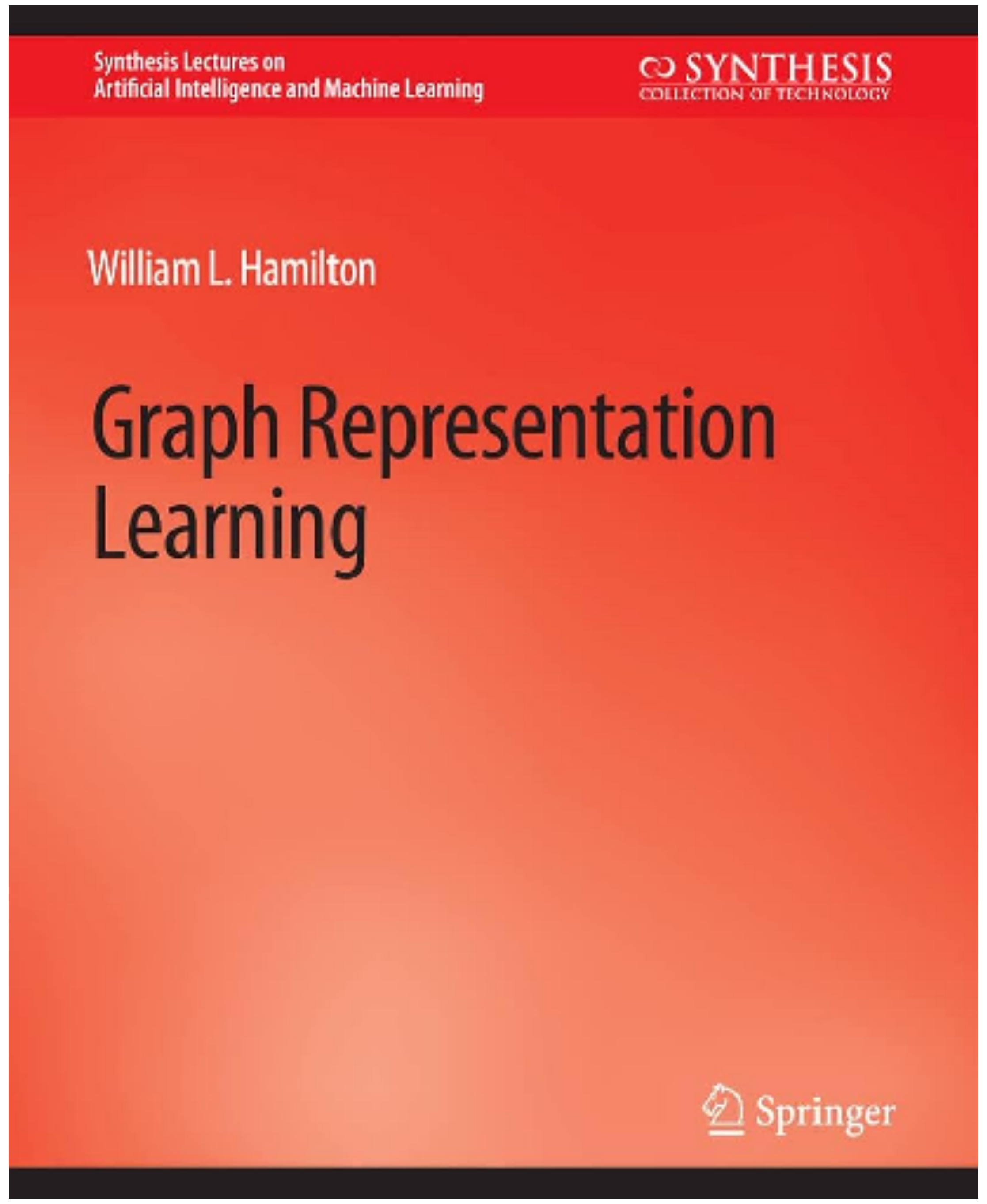
https://www.dgl.ai/

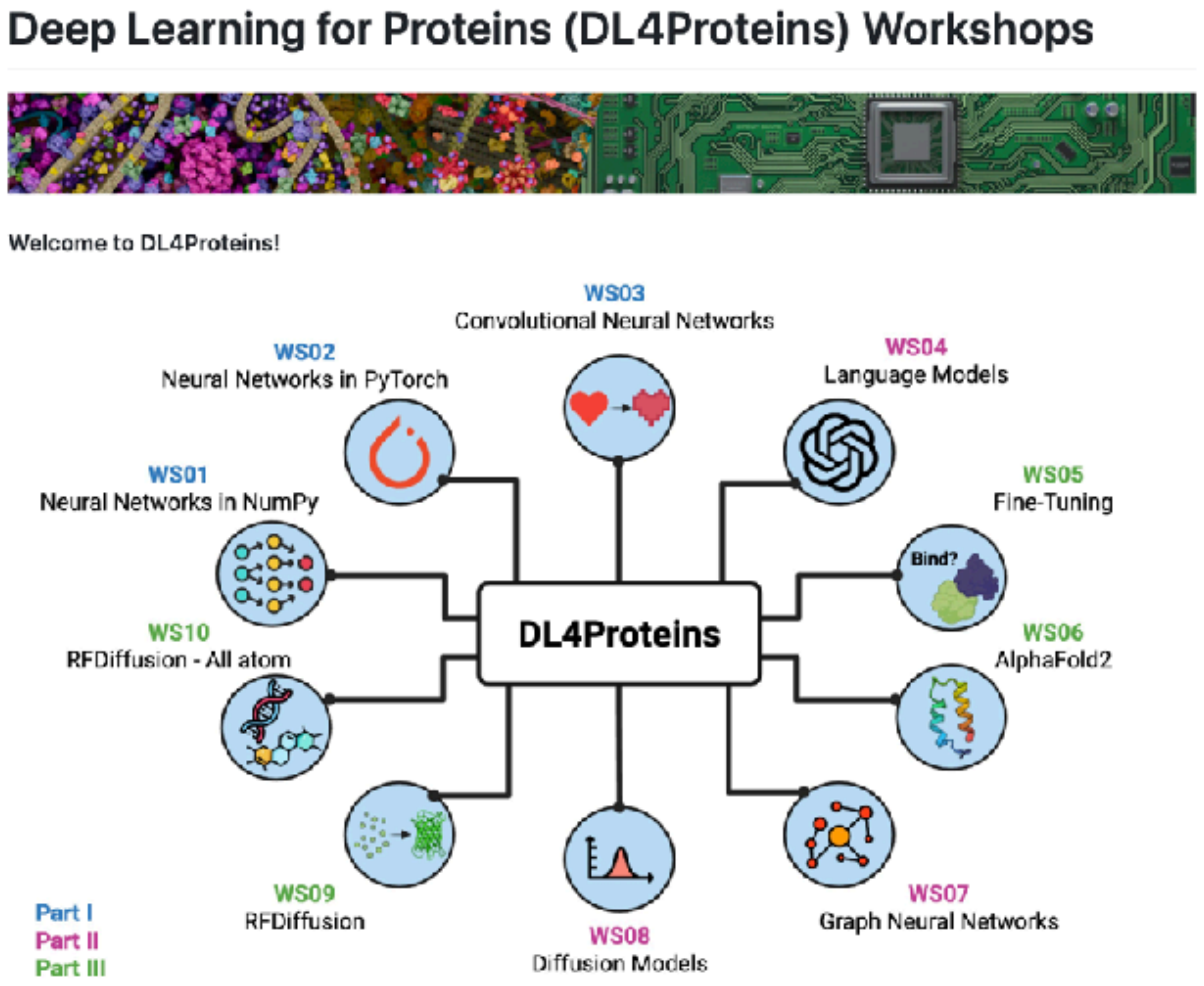
https://graphneural.network/

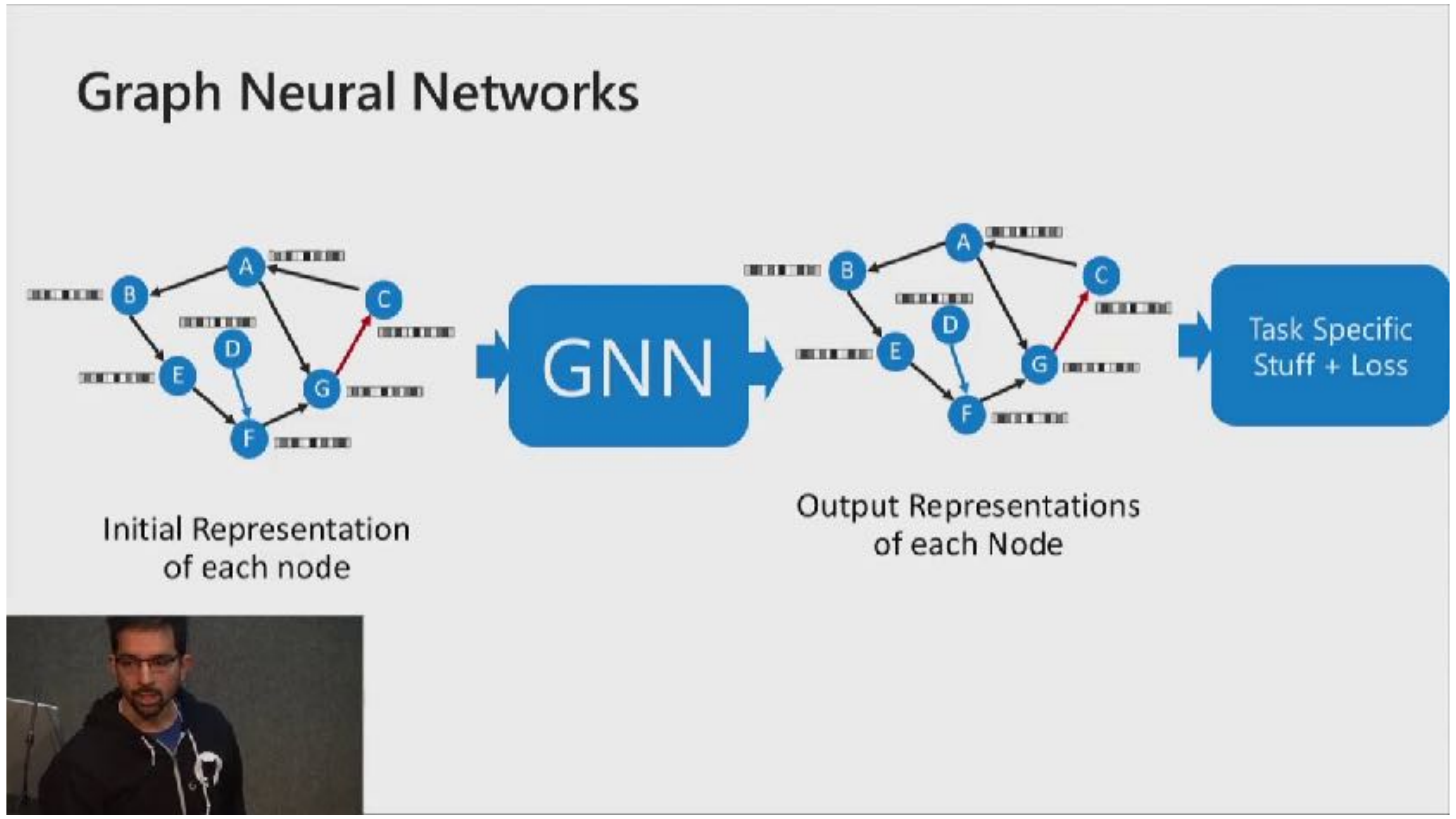
Jraph

https://github.com/google-deepmind/jraph

References







https://www.cs.mcgill.ca/~wlh/grl_book/files/GRL_Book.pdf

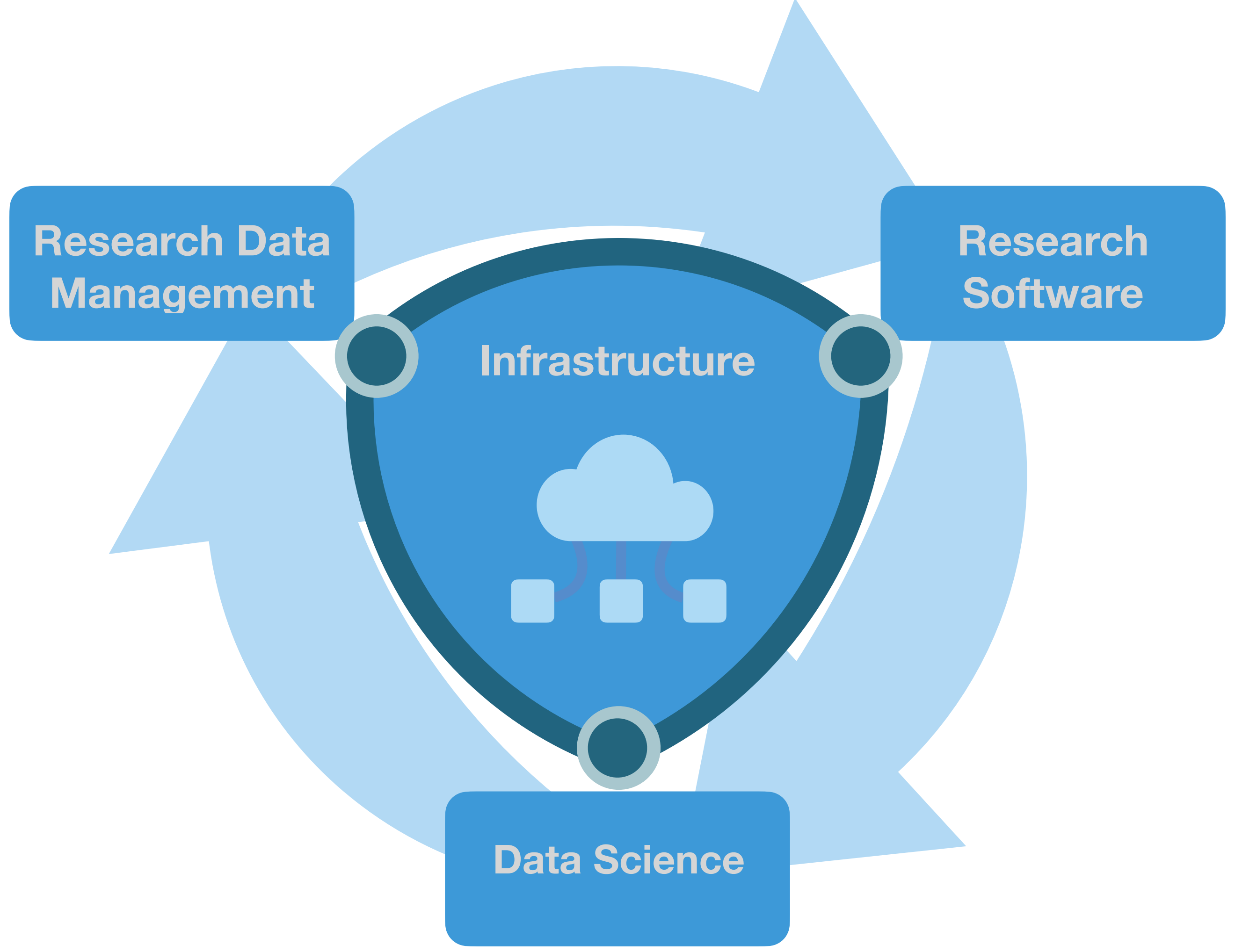
https://github.com/Graylab/DL4Proteins-notebooks

https://www.youtube.com/watch?v=zCEYiCxrL_0

Thank you

Multi-omics Network Analytics Research Group

Informatics Platform



DTU Health Tech
Department of Health Technology

albsad@dtu.dk

@albsantosdel

https://github.com/Multiomics-Analytics-Group

https://multiomics-analytics-group.github.io/

DTU

BRiGHT

PIONEER CENTRE FOR ARTIFICIAL INTELLIGENCE